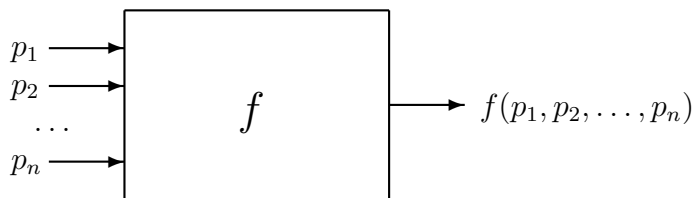


Boolean functions

A **Boolean function f of n arguments** is a function that,

- for every (input) sequence $\boxed{p_1, p_2, \dots, p_n}$ of **0**s and **1**s,
- returns (outputs) a **value** $\boxed{f(p_1, p_2, \dots, p_n)}$ that is also either **0** or **1**.



In other words, an **n -ary Boolean function f** is a function whose

- **domain** is the set $\{0, 1\}^n$ of all n -tuples of **0**s and **1**s, and
- **co-domain** is the set $\{0, 1\}$.

In symbols: $\boxed{f : \{0, 1\}^n \rightarrow \{0, 1\}}$

Truth-tables: a way of describing Boolean functions

A **truth-table** for an n -ary Boolean function $f(p_1, \dots, p_n)$ consists of two parts:

- The left part contains n columns, where all 2^n input n -tuples of **0**s and **1**s are listed as possible values for the propositional (Boolean) variables $p_1, \dots, p_n$.
- The right part contains a single column, giving the value $f(p_1, \dots, p_n)$ for each corresponding input n -tuple.

p_1	p_2	$\dots$	p_n	$f(p_1, p_2, \dots, p_n)$
0	0	$\dots$	0	$f(0, 0, \dots, 0)$
0	0	$\dots$	1	$f(0, 0, \dots, 1)$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
1	1	$\dots$	0	$f(1, 1, \dots, 0)$
1	1	$\dots$	1	$f(1, 1, \dots, 1)$

Exercise: How many different n -ary Boolean functions exist?

Boolean functions of one argument

There are $2^{2^1} = 4$ unary (=1-ary) Boolean functions, f_1 , f_2 , f_3 , and f_4 :

p	$f_1(p)$
0	0
1	0

p	$f_2(p)$
0	1
1	1

p	$f_3(p)$
0	0
1	1

p	$f_4(p)$
0	1
1	0

- $f_1(p) = 0$ is the constant 0 function (doesn't depend on the value of p).
- $f_2(p) = 1$ is the constant 1 function (doesn't depend on the value of p).
- $f_3(p) = p$ always (it is called the identity function).
- $f_4(p) = \neg p$ (negation).

Familiar Boolean functions of two arguments

p	q	$f_1(p, q)$
0	0	0
0	1	0
1	0	0
1	1	1

p	q	$f_2(p, q)$
0	0	0
0	1	1
1	0	1
1	1	1

p	q	$f_3(p, q)$
0	0	1
0	1	1
1	0	0
1	1	1

p	q	$f_4(p, q)$
0	0	1
0	1	0
1	0	0
1	1	1

- $f_1(p, q) = p \wedge q$ (conjunction).
- $f_2(p, q) = p \vee q$ (disjunction).
- $f_3(p, q) = p \rightarrow q$ (implication).
- $f_4(p, q) = p \leftrightarrow q$ (bi-implication).

Boolean functions of two arguments are also called binary Boolean functions.

Some more binary Boolean functions

p	q	$f_5(p, q)$
0	0	0
0	1	1
1	0	1
1	1	0

p	q	$f_6(p, q)$
0	0	1
0	1	1
1	0	1
1	1	0

p	q	$f_7(p, q)$
0	0	1
0	1	0
1	0	0
1	1	0

p	q	$f_8(p, q)$
0	0	0
0	1	0
1	0	1
1	1	1

- $f_5(p, q) = (p \vee q) \wedge \neg(p \wedge q)$ is the **exclusive or**. Notation: $p \oplus q$
- $f_6(p, q) = \neg(p \wedge q)$ is called the **Scheffer stroke (nand)**.
- $f_7(p, q) = \neg(p \vee q)$ is called the **Pierce arrow (nor)**.
- $f_8(p, q) = p$ (it always 'selects' p and doesn't depend on q).

Exercise: Are there some more binary Boolean functions?

$\wedge$, $\vee$, and $\neg$ are functionally complete

Every Boolean function of n arguments can be represented by a formula using propositional variables and connectives $\wedge$, $\vee$, and $\neg$ ONLY.

For example:

x_1	x_2	x_3	$f(x_1, x_2, x_3)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Each 3-tuple in the left part of the table can be represented as a conjunction. E.g.:

0 0 1 is represented by $\neg x_1 \wedge \neg x_2 \wedge x_3$

1 0 0 is represented by $x_1 \wedge \neg x_2 \wedge \neg x_3$

1 1 1 is represented by $x_1 \wedge x_2 \wedge x_3$

The function $f(x_1, x_2, x_3)$ can be represented as a disjunction of the conjunctions that correspond to the rows where $f(x_1, x_2, x_3)$ is **1**:

$$(\neg x_1 \wedge \neg x_2 \wedge x_3) \vee (x_1 \wedge \neg x_2 \wedge \neg x_3) \vee (x_1 \wedge x_2 \wedge x_3)$$

Logically equivalent formulas represent the **same** Boolean function.

The need for a compact representation

- Truth-tables are **too large**: to represent $f(x_1, \dots, x_n)$ we need 2^n lines.
- Formulas are more **compact**. But, say, to decide whether a given formula is *valid*, we may have to go through all the **exponentially many** lines of its truth-table.
- **Tableaux** can also have **exponentially many** cases.

It is not known whether there are better representations that work in *every* case. Still, it makes sense to look for them, and explore options. So, next we are after:

- **Compact representations** for Boolean functions using

Boolean decision diagrams (BDDs).

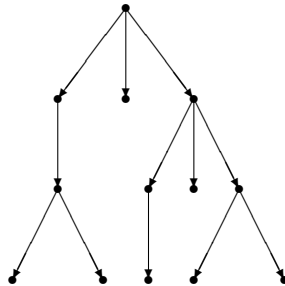
- How various logical operations can be **effectively implemented** using this representation.
- (Later on during the course, BDDs will be applied to program verification.)

Directed trees

A **directed tree** is a directed graph in which:

- There is a *unique* node with no edges going into it: the **initial node** or **root**.
- There is always a *unique* path from the root to every other node in the tree. We call such a path a **full path**.

We usually draw a directed tree with its root at the top:

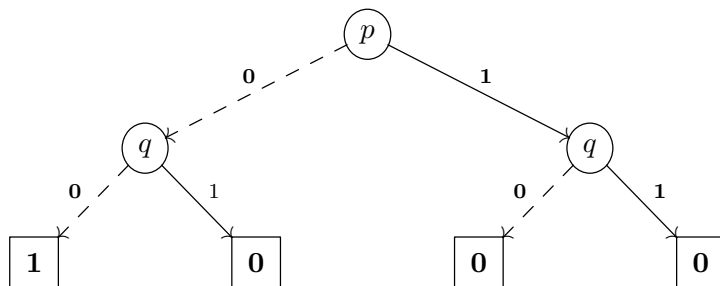


- A childless node is called a **leaf** or a **terminal node**.
- Nodes with at least one child are called **internal** or **non-terminal nodes**.

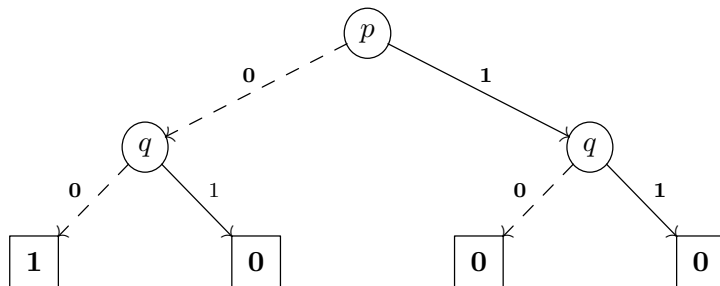
Binary decision trees

A binary decision tree is a directed tree in which:

- Each non-terminal node is labelled with a propositional variable.
- Each terminal node is labelled with either **0** or **1**.
- For each non-terminal node n , there are exactly two edges leaving n :
 - one (**dashed**) labelled with **0**, leading to a node denoted by $lo(n)$
 - another (**solid**) labelled with **1**, leading to a node denoted by $hi(n)$
- **The same propositional variable cannot occur as a label twice in any path.**
(But it might occur again in another path.)



Binary decision trees represent Boolean functions



This tree represents the same Boolean function as the formula $\neg(p \vee q)$:

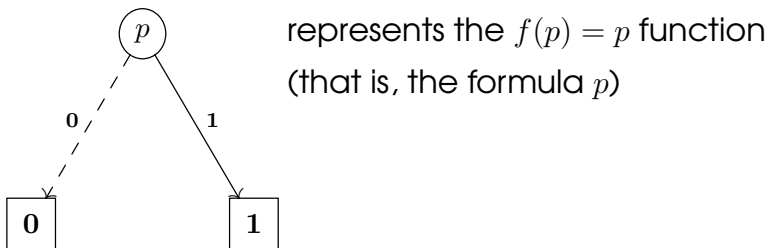
p	q	$\neg(p \vee q)$
0	0	1
0	1	0
1	0	0
1	1	0

Each line in the truth-table for $\neg(p \vee q)$ corresponds to a *full path* in the binary decision tree and vice versa.

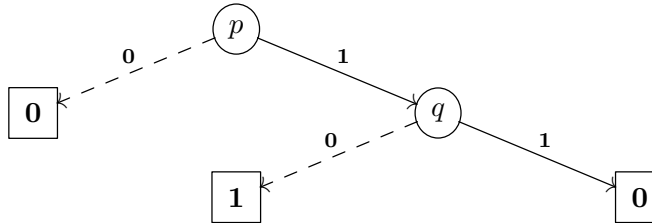
Binary decision trees for the simplest Boolean functions

0 represents the constant **0** function (that is, the formula **0**)

1 represents the constant **1** function (that is, the formula **1**)



More compact binary decision trees



This tree represents the same Boolean function as the formula $p \wedge \neg q$:

p	q	$p \wedge \neg q$
0	0	0
0	1	0
1	0	1
1	1	0

Two lines in the truth-table are represented by one full path $(p) \xrightarrow{0} [0]$.

Optimising binary decision trees

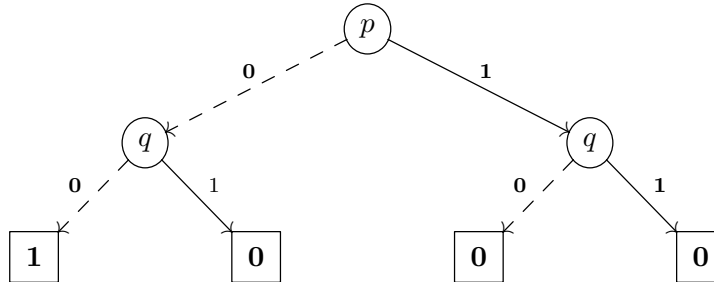
For any given Boolean function, the size of a full binary decision tree representing it is close to the size of its truth-table: an n -ary Boolean function has 2^n lines in its truth-table, corresponding to 2^n full paths in the tree.

However, binary decision trees often contain some redundancy which we can exploit. We will consider three kinds of optimisation techniques:

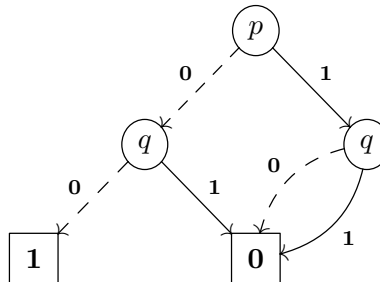
- removing duplicate terminals,
- removing redundant tests,
- removing duplicate non-terminals.

Optimisation I: removing duplicate terminals

Consider again the binary decision tree representing $\neg(p \vee q)$:



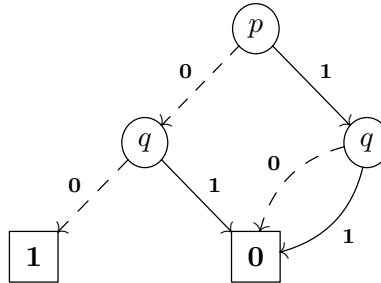
We can 'optimise' it by gluing together all terminal nodes labelled with **0**, and all terminal nodes labelled with **1**:



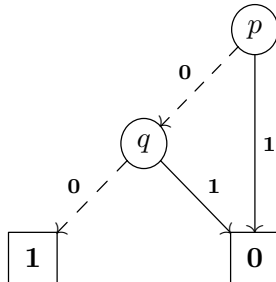
This binary decision 'diagram' (BDD) still represents $\neg(p \vee q)$.

(Though, it is no longer a tree.)

Optimisation II: removing redundant tests

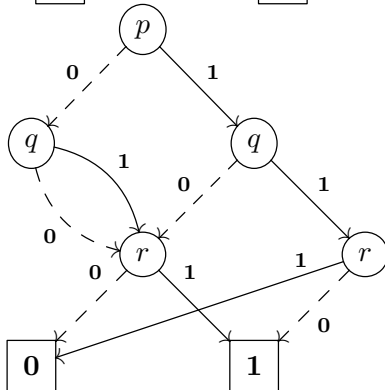
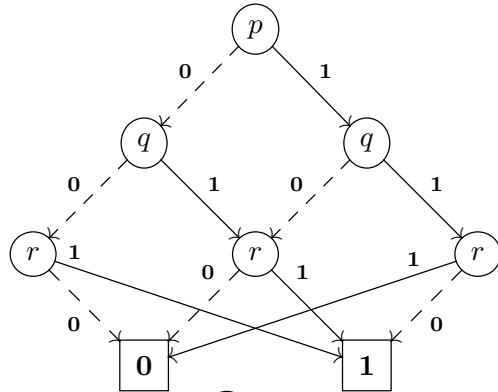


The right q -node above is unnecessary, because we go to the same place whether it is **0** or **1**. So we eliminate it, sending its incoming edge to **0**:



This BDD still represents $\neg(p \vee q)$ and has fewer nodes.

Optimisation III: removing duplicate non-terminals

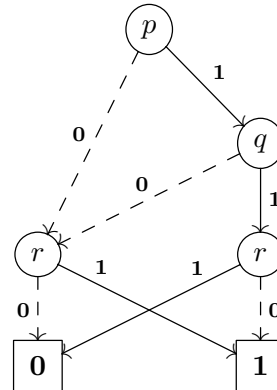


The two left-most r -nodes are 'duplicates' (they 'do the same thing').

We eliminate one of them and redirect its incoming edge(s) to the other one.

Then eliminate the redundant q -test by performing an 'Optimisation II-type' step:

$\leadsto$



Binary decision diagrams (BDDs)

- A directed acyclic graph (dag) is a directed graph without directed cycles.
- A node n of a dag is an initial node or root, if there are no edges going into n .
- A node n is a terminal node, if there are no edges going out of n .
All other nodes are called non-terminal.

A binary decision diagram (BDD) is a finite dag such that:

- It has a unique initial node.
- All of its terminal nodes are labelled with **0** or **1**, and all non-terminal nodes are labelled with propositional variables.
- Each non-terminal node has exactly two edges from that node to others:
 - one — **dashed** — is labelled with **0**, leading to a node denoted by $lo(n)$,
 - one — **solid** — is labelled with **1**, leading to a node denoted by $hi(n)$.

(We might omit the edge-labels, since being dashed/solid uniquely identifies them.)

- **The same propositional variable cannot occur as a label twice in any path.**

Binary decision trees are special cases of BDDs.

Optimising BDDs leads to reduced BDDs

Summing up, there are three ways of reducing a BDD to a more compact form:

- **Removal of duplicate terminals:**

If a BDD contains more than one terminal **0**-node, then we redirect all edges which go to any **0**-node to just one of them. We proceed the same way with terminal **1**-nodes.

- **Removal of redundant tests:**

If both outgoing edges from a node n go to the same node m , then we eliminate n and redirect all its incoming edges to m .

- **Removal of duplicate non-terminals:**

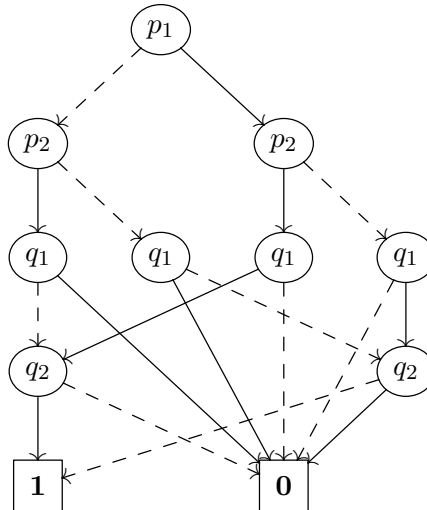
If 'below' two distinct non-terminal nodes n and m there are BDDs of identical structure (that is, isomorphic as graphs), then

- we eliminate n ,
- we eliminate those nodes and edges that are no longer reachable from the root, and
- redirect all the incoming edges of n to m .

A BDD is called **reduced** if *none* of these rules can be applied to it (that is, *no* more reductions are possible).

Exercise 3.1

Find a formula which is represented by the following reduced BDD, and built up using logical connectives from $\neg$, $\vee$, and $\wedge$ only.

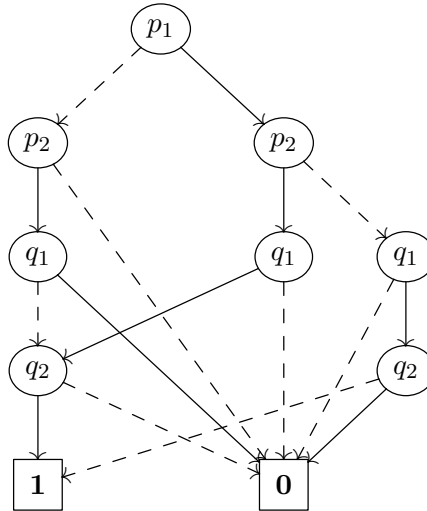


SOLUTION: Describe the paths leading from the root to **1**, using $\neg$, $\vee$, and $\wedge$ only:

$$(\neg p_1 \wedge p_2 \wedge \neg q_1 \wedge q_2) \vee (\neg p_1 \wedge \neg p_2 \wedge \neg q_1 \wedge \neg q_2) \vee (p_1 \wedge p_2 \wedge q_1 \wedge q_2) \vee (p_1 \wedge \neg p_2 \wedge q_1 \wedge \neg q_2)$$

Exercise 3.2

Find a formula which is represented by the following reduced BDD, and built up using logical connectives from $\neg$, $\rightarrow$, and $\wedge$ only.



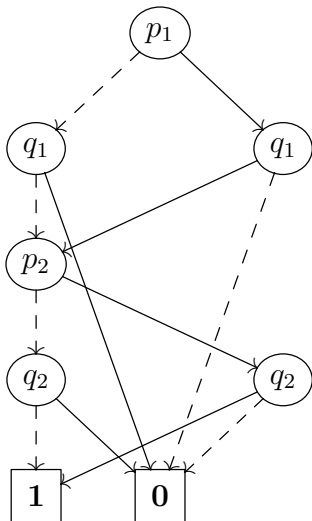
SOLUTION: Describe the paths leading from the root to **1** , using $\neg$, $\rightarrow$, and $\wedge$ only:

$$\left(\neg p_1 \rightarrow (p_2 \wedge \neg q_1 \wedge q_2) \right) \wedge \left(p_1 \rightarrow (p_2 \rightarrow (q_1 \wedge q_2)) \right) \wedge \left(p_1 \rightarrow (\neg p_2 \rightarrow (q_1 \wedge \neg q_2)) \right)$$

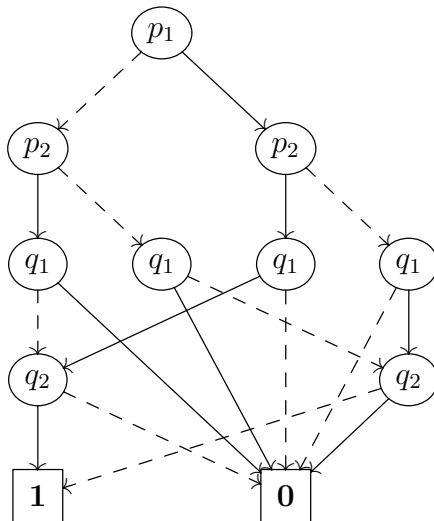
Ordered BDDs: size matters

When we draw a reduced BDD for a formula, it does matter in which **order** we start considering its variables. Here are reduced BDDs for $(p_1 \leftrightarrow q_1) \wedge (p_2 \leftrightarrow q_2)$, with two different variable orderings:

With p_1, q_1, p_2, q_2 :



With p_1, p_2, q_1, q_2 :



Ordered BDDs

Let $(x_1, \dots, x_n)$ be an ordered list of propositional variables without duplicates. We say that a BDD B **is with ordering** $(x_1, \dots, x_n)$ if

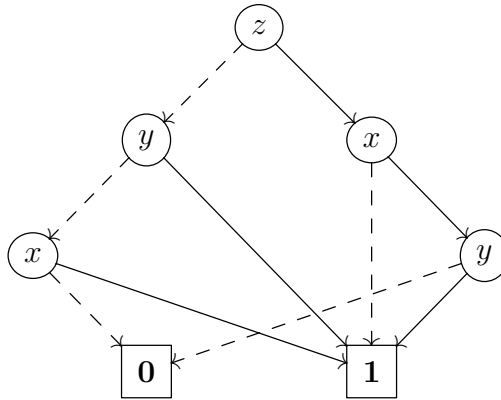
- all variables occurring in B are among $x_1, \dots, x_n$, and
- along **every** path in B , if x_i is before x_j on the path, then x_i is before x_j in the ordering.

(In other words: When we start drawing B , we consider the variables according to the order $x_1, \dots, x_n$.)

An **ordered BDD (OBDD)** is a BDD which is having some ordering of variables.

- The size of a BDD may depend **critically** on the variable ordering in it (see previous slide).
- **Not every** BDD is ordered (see next slide).

A reduced BDD without an ordering of variables



It is a BDD, as no variable occurs twice along any path. But it is NOT ordered:

- On the leftmost (all-dashed) path, y comes before x .
- On the rightmost (all-solid) path, x comes before y .

Exercise 3.3

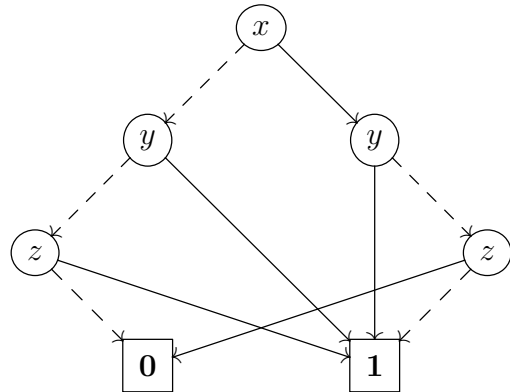
Construct a reduced OBDD, using the variable ordering (x, y, z) , that represents the same Boolean function as the BDD on the previous slide.

SOLUTION:

Truth-table of the function:

x	y	z	$f(x, y, z)$
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Reduced OBDD with ordering (x, y, z) :

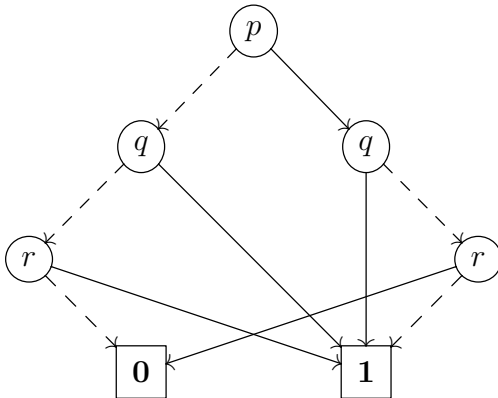


Compatible variable orderings

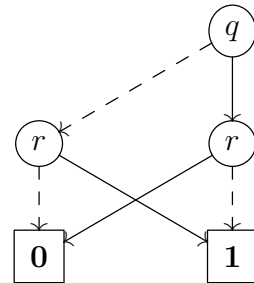
The orderings of two OBDDs B_1 and B_2 are said to be compatible, if there are *no* variables x and y such that

- x comes before y in the ordering of B_1 , but
- y comes before x in the ordering of B_2 .

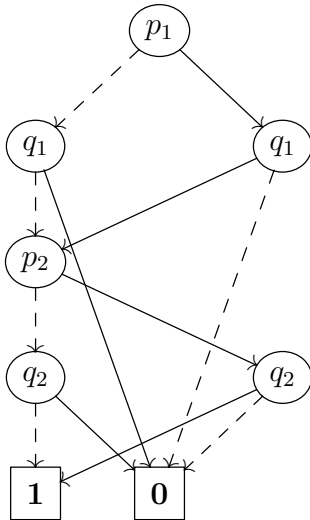
FOR EXAMPLE:



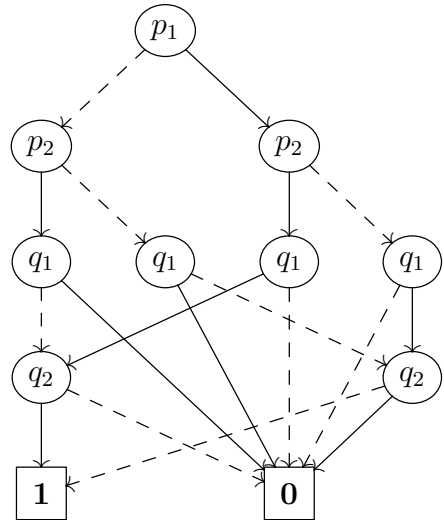
and



Incompatible variable orderings: an example



and



Important notes on reduced OBDDs

- Given a variable ordering, the reduced OBDD representing a given Boolean function is **unique**. In other words, if B_1 and B_2 are reduced OBDDs with compatible variable orderings that represent the same Boolean function, then they have **identical structure** (that is, isomorphic as graphs).
- Several **heuristics** have been developed for finding a good variable ordering when such an ordering exists.
- However, there are Boolean functions that have **exponential size** OBDDs for **every** variable ordering.

An example is the following $2n$ -ary Boolean function:

$$f(a_1, \dots, a_n, b_1, \dots, b_n) = n^{\text{th}} \text{ bit of the } 2n\text{-bit binary integer } (a_1 \cdots a_n) \cdot (b_1 \cdots b_n).$$

- Absence of redundant variables:**

If the value of the Boolean function $f(p_1, \dots, p_n)$ does not depend on the value of p_i , then **no** reduced OBDD representing f contains **any** p_i -node.

Decision algorithms by reduced OBDDs

- **Deciding validity:**

A formula A is valid iff the reduced OBDD for A is $\boxed{1}$.

- **Deciding satisfiability:**

A formula A is satisfiable iff the reduced OBDD for A is **not** $\boxed{0}$.

- **Deciding logical equivalence:**

Two Formulas A and B are equivalent iff the reduced OBDDs for A and B *with compatible variable orderings* have **identical structure** (isomorphic as graphs).

- **Deciding logical consequence:**

Formulas $A_1, \dots, A_n$ logically imply formula B iff the reduced OBDD for the formula $(A_1 \wedge \dots \wedge A_n \wedge \neg B)$ is $\boxed{0}$.

Algorithms on OBDDs

OBDDs are useful: Once we have found 'small' OBDDs for our Boolean functions (formulas), then manipulating them is **fast**.

We will discuss three algorithms on reduced OBDDs:

- apply,
- restrict,
- exists.

These are basic algorithms used in *program verification*.

The algorithm `apply`

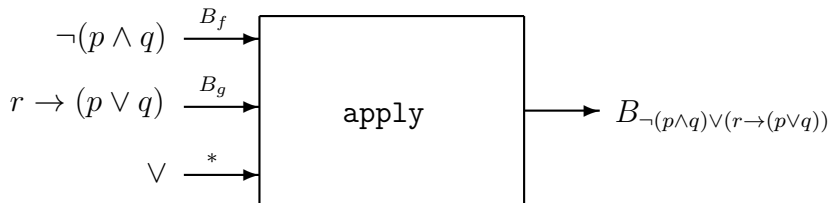
Suppose we have two reduced OBDDs B_f and B_g , representing the Boolean functions f and g such that their variable orderings are **compatible**.

Then `apply`($*$, B_f , B_g) **computes the reduced OBDD** B_{f*q} representing

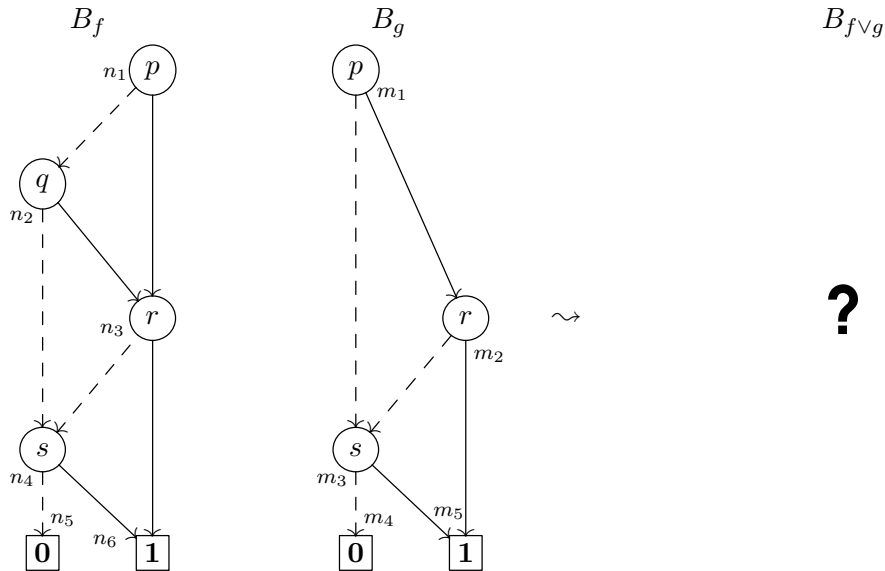
$$f * g,$$

where $*$ can be any of $\vee, \wedge, \rightarrow, \leftrightarrow$.

For example:



Example: $\text{apply}(\vee, B_f, B_g)$



Initial steps:

- We determine the compatible variable ordering: (p, q, r, s) .
- We name the nodes in each input-OBDD (to be able to refer to them).

apply: **general description**

We build B_{f*g} **recursively step by step**, starting from its **root**.

At each and every step, we compare two nodes:

- one from B_f (denoted by $\underline{n_f}$), and
- one from B_g (denoted by $\underline{n_g}$).

Then proceed according to one of the four cases, described on slides 89–90 below. We stop when we have reached the terminal nodes on every path.

The resulting OBDD B_{f*g} is not necessarily *reduced*. **Finally**, we have to apply the known **reductions** to always end up with a reduced OBDD.

Alternatively, we can also do reductions **during** the comparison process.

Comparing a pair (n_f, n_g) : the first two cases

We have a look at their labels: $\underline{\ell(n_f)}$ and $\underline{\ell(n_g)}$.

Depending on these labels, there are four cases:

- **Case 1.**

If both nodes n_f and n_g are terminal nodes (that is, labelled by **1** or **0**), then:

- We compute the truth-value $\ell(n_f) * \ell(n_g)$ (using the truth-table of $*$), and
- create a terminal node labelled by the result.

- **Case 2.**

If both n_f and n_g are labelled by the **same** variable, say, p , then:

- We create a new node labelled by p ,
- with a dashed line to comparing $(lo(n_f), lo(n_g))$, and
- with a solid line to comparing $(hi(n_f), hi(n_g))$.

Comparing a pair (n_f, n_g) : the remaining two cases

- **Case 3:**

If n_f is a non-terminal node, and n_g is either terminal or $\ell(n_f)$ comes before $\ell(n_g)$ in the variable ordering, then:

- We create a new node labelled by $\ell(n_f)$, the 'smaller' label,
- with a dashed line to comparing $(lo(n_f), n_g)$, and
- with a solid line to comparing $(hi(n_f), n_g)$.

- **Case 4:**

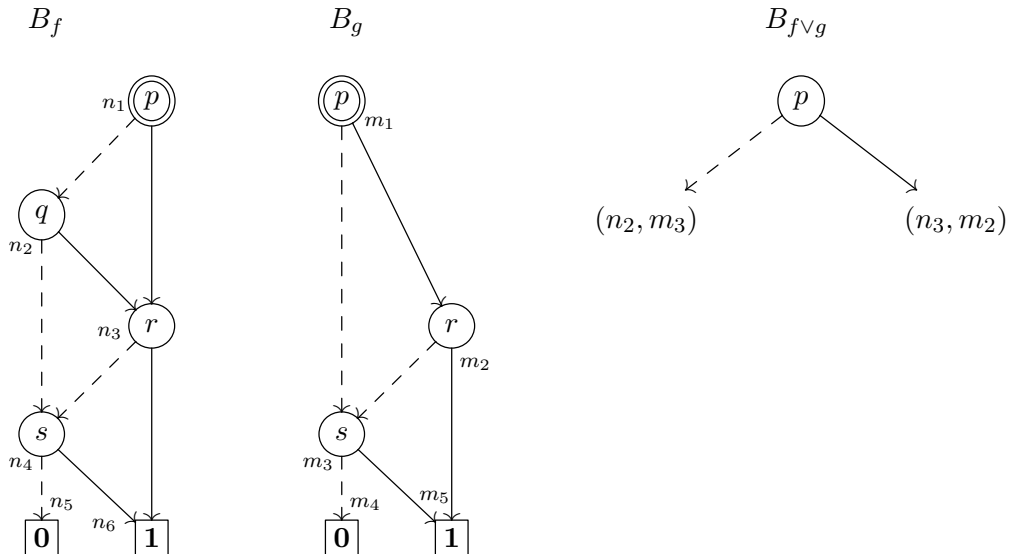
If n_g is a non-terminal node, and n_f is either terminal or $\ell(n_g)$ comes before $\ell(n_f)$ in the variable ordering, then:

- We create a new node labelled by $\ell(n_g)$, the 'smaller' label,
- with a dashed line to comparing $(n_f, lo(n_g))$, and
- with a solid line to comparing $(n_f, hi(n_g))$.

The first step in $\text{apply}(\vee, B_f, B_g)$: comparing (n_1, m_1)

We always begin with comparing the roots:

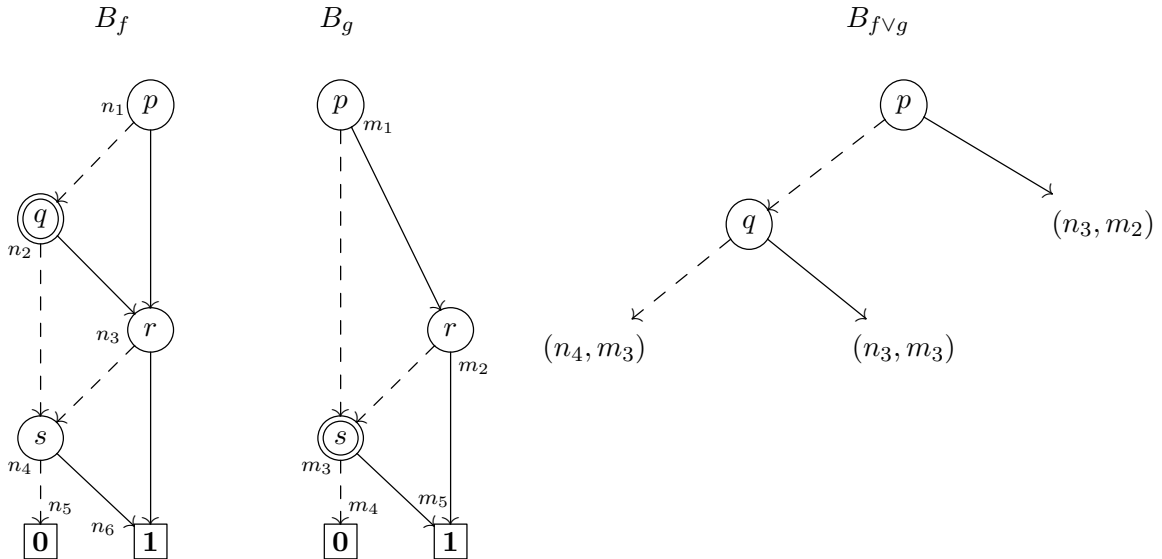
Case 2 applies: $\ell(n_1) = \ell(m_1) = p$.



We continue with the remaining comparisons in an arbitrary order.

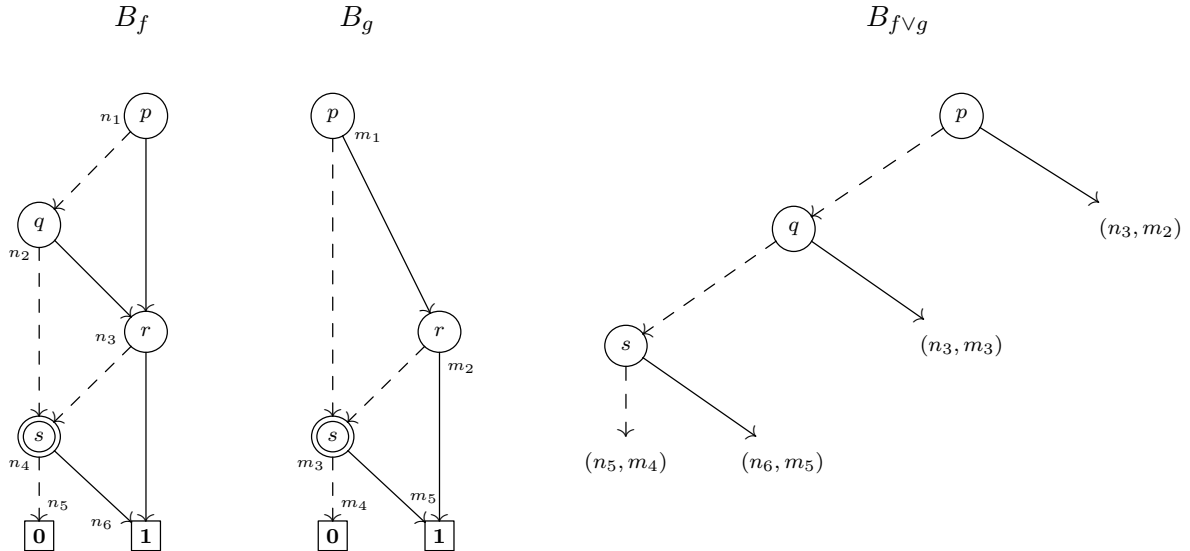
apply($\vee, B_f, B_g$): **comparing** (n_2, m_3)

Case 3 applies: $\ell(n_2) = q$ comes before $\ell(m_3) = s$ in the ordering.



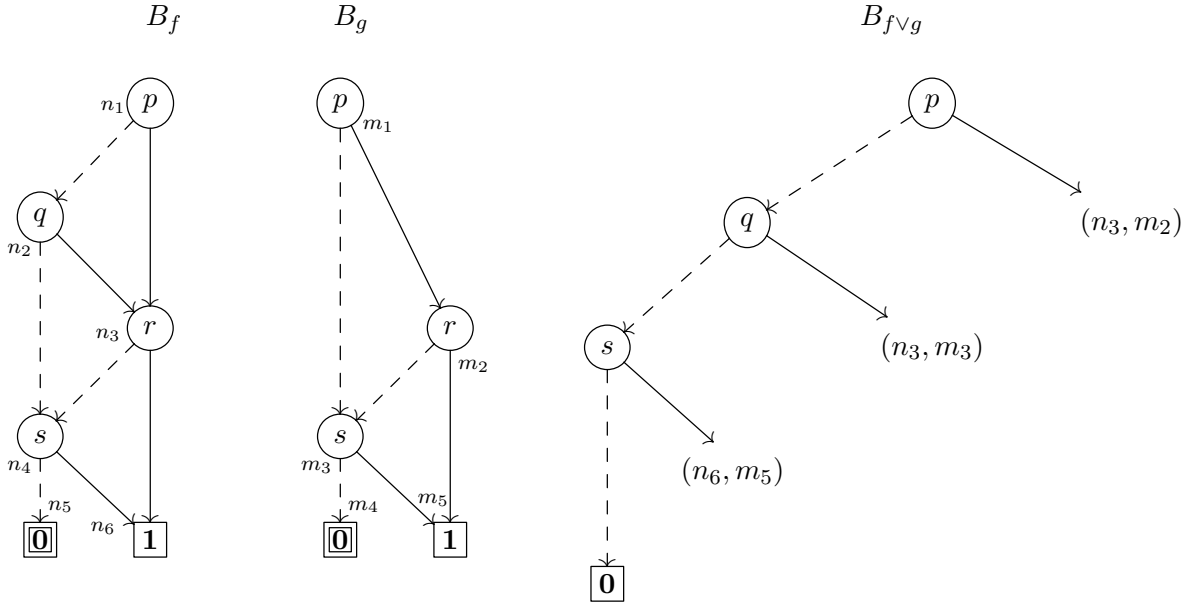
apply($\vee, B_f, B_g$): **comparing** (n_4, m_3)

Case 2 applies again: $\ell(n_4) = \ell(m_3) = s$.



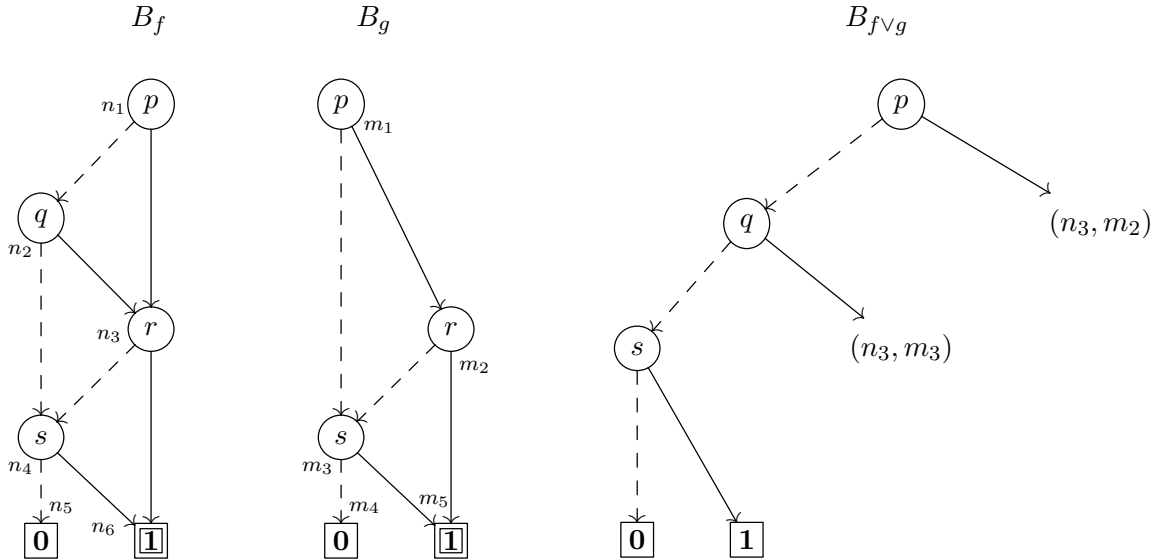
apply($\vee, B_f, B_g$): **comparing** (n_5, m_4)

Case 1 applies: $\ell(n_5) = \mathbf{0}$ and $\ell(m_4) = \mathbf{0}$. As $\mathbf{0} \vee \mathbf{0} \equiv \mathbf{0}$, we have $\boxed{\mathbf{0}}$ as new node.



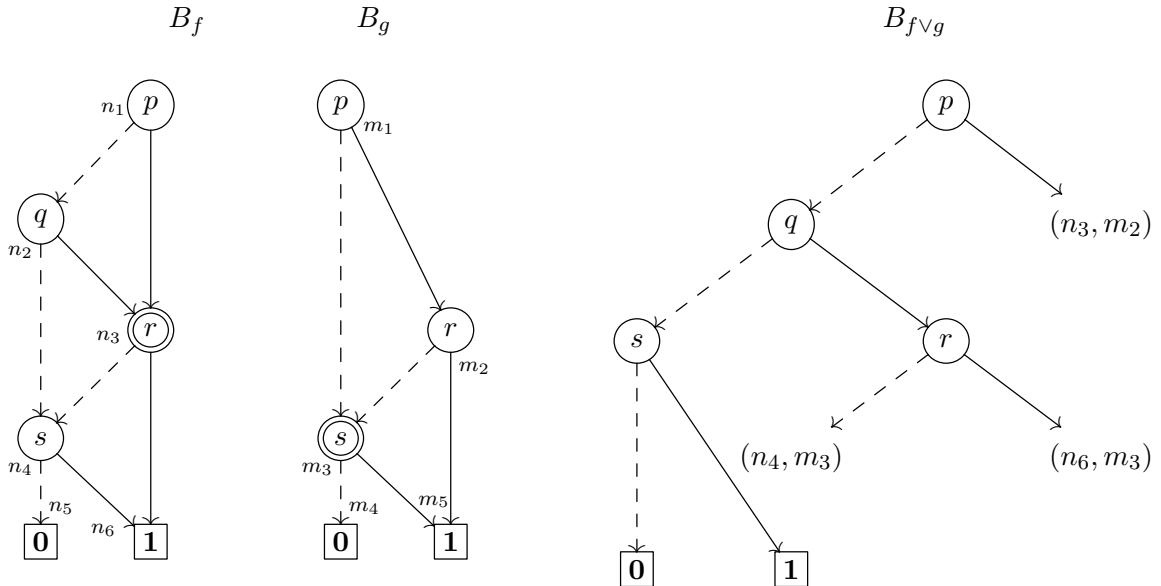
apply($\vee, B_f, B_g$): **comparing** (n_6, m_5)

Case 1 applies: $\ell(n_6) = 1$ and $\ell(m_5) = 1$. As $1 \vee 1 \equiv 1$, we have $\boxed{1}$ as new node.



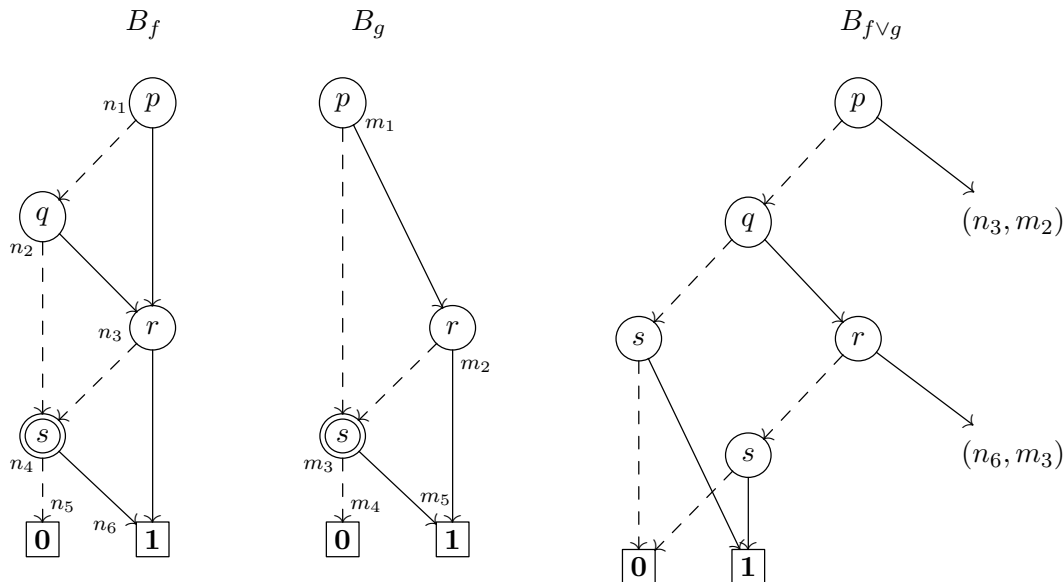
apply($\vee, B_f, B_g$): **comparing** (n_3, m_3))

Case 3 applies: $\ell(n_3) = r$ comes before $\ell(m_3) = s$ in the ordering.



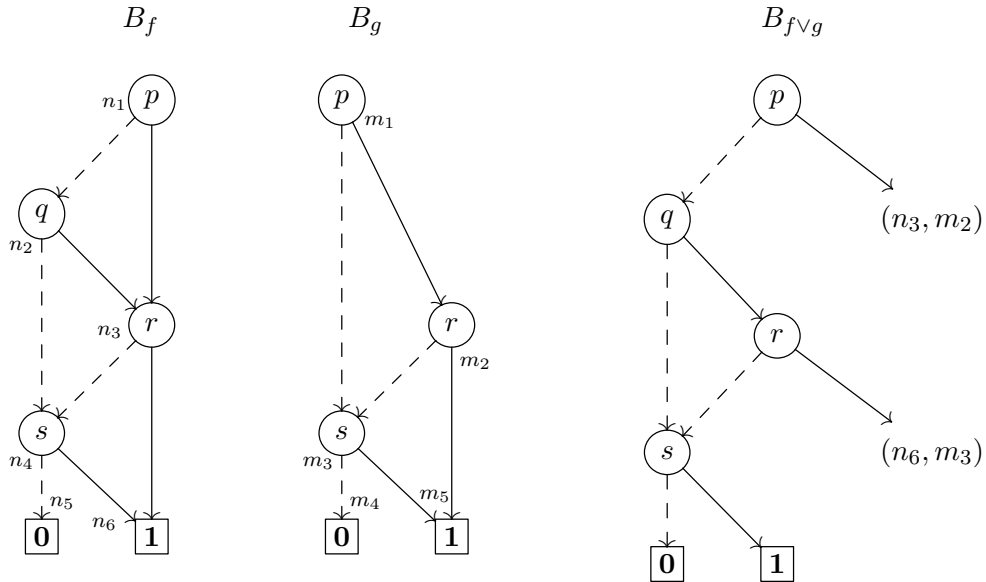
apply($\vee$, B_f , B_g): comparing (n_4, m_3) again

We already know the result: see slides 93–95:



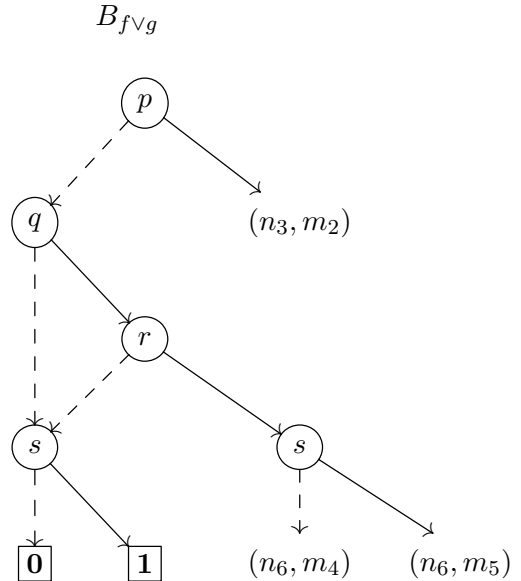
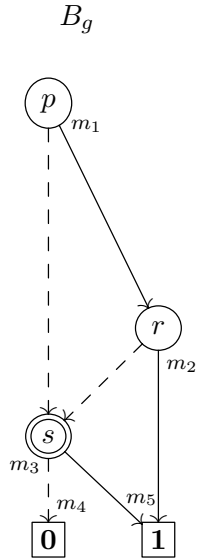
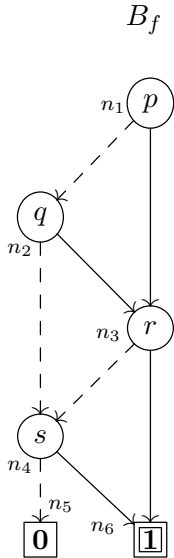
Observe that we also did reductions, by *not creating new terminal nodes*.

$\text{apply}(\vee, B_f, B_g)$: removing duplicate non-terminals



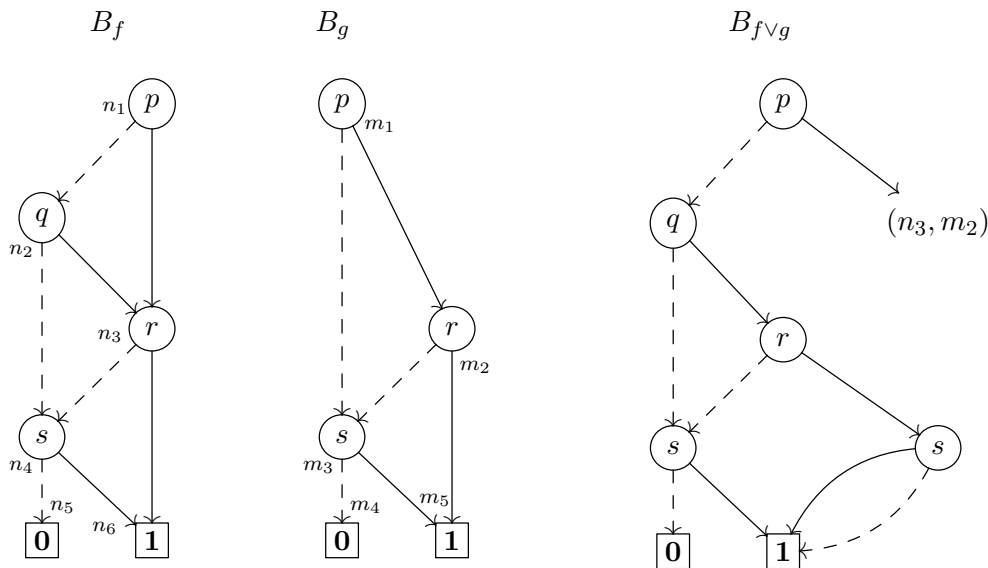
apply($\vee, B_f, B_g$): **comparing** (n_6, m_3))

Case 4 applies: n_6 is terminal and m_3 is non-terminal.



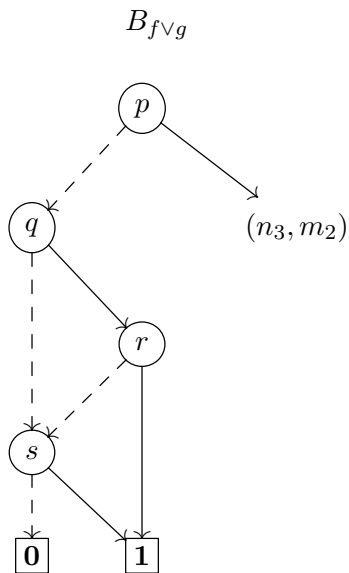
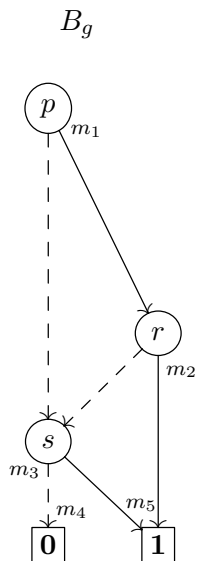
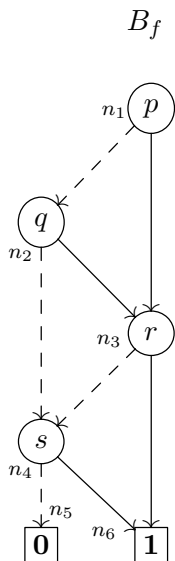
$\text{apply}(\vee, B_f, B_g)$: **comparing** (n_6, m_4) , and then (n_6, m_5)

Case 1 applies for both: $1 \vee 0 \equiv 1$ and $1 \vee 1 \equiv 1$.



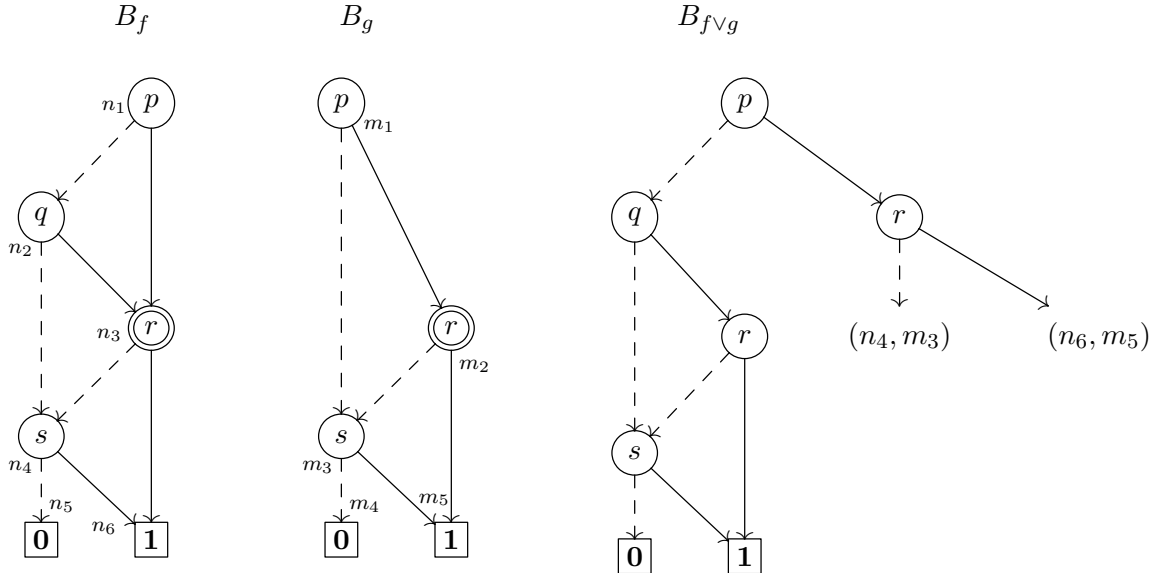
Again, we reduced the size by *not creating new terminal nodes*.

$\text{apply}(\vee, B_f, B_g)$: removing a redundant test



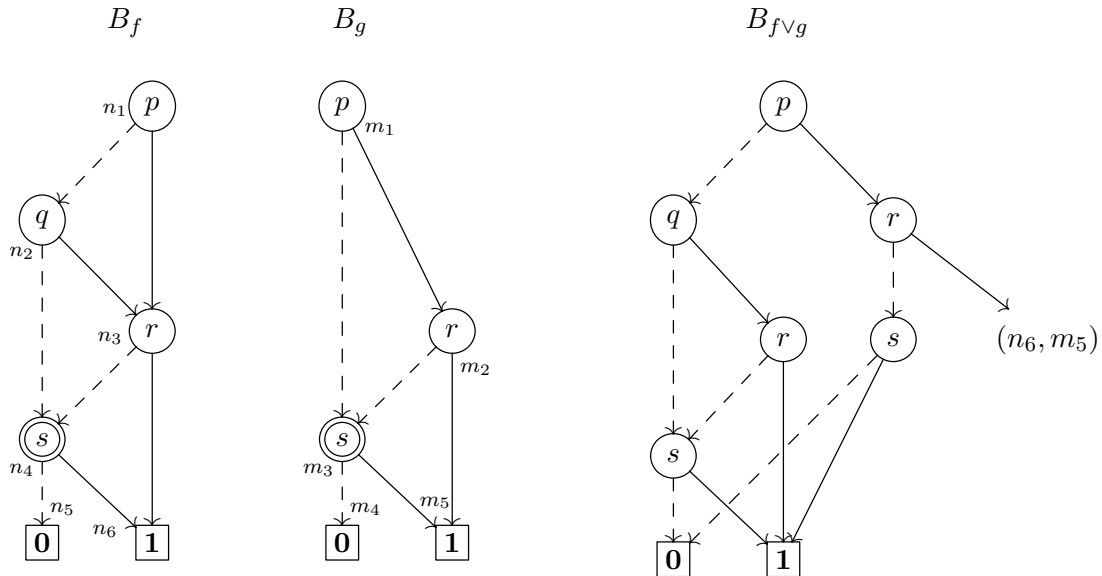
apply($\vee, B_f, B_g$): **comparing** (n_3, m_2)

Case 2 applies again: $\ell(n_3) = \ell(m_2) = r$.



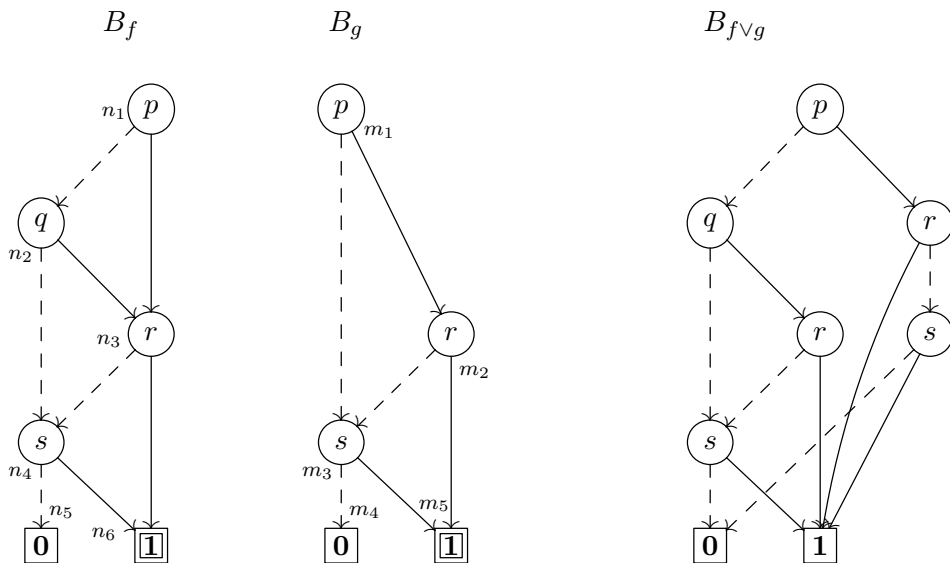
apply($\vee, B_f, B_g$): comparing (n_4, m_3) yet again

We already know the result, see slides 93–95 again:



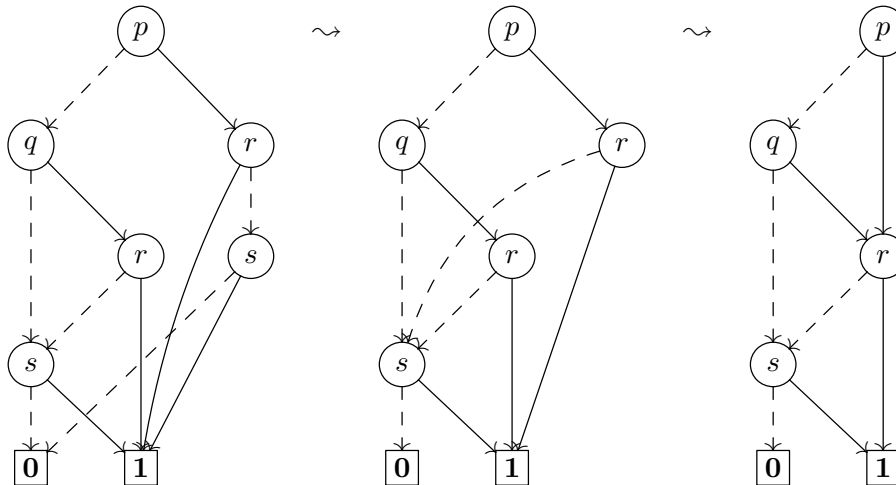
apply($\vee, B_f, B_g$): **comparing** (n_6, m_5) **again**

We already know the result, see slide 100 (or directly, $1 \vee 1 \equiv 1$):



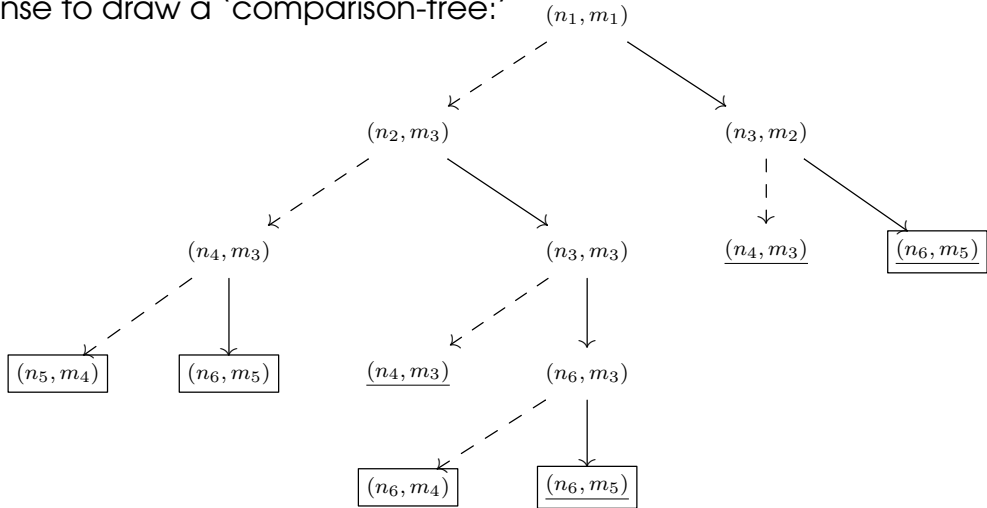
apply($\vee, B_f, B_g$): final reduction steps

$\text{apply}(\vee, B_f, B_g) = B_{f \vee g}$



apply($\vee, B_f, B_g$): summary

It makes sense to draw a 'comparison-tree':



- It helps us to detect multiple occurrences (underlined, like (n4, m3)).
- Exercise: Compute $\text{apply}(\rightarrow, B_f, B_g)$.

(Hint: How can the comparison-tree help?)

Restrictions of a formula

Let A be formula and p a propositional variable. Then:

- $A[0/p]$ is the formula obtained by replacing all occurrences of p in A with **0**.
- $A[1/p]$ is the formula obtained by replacing all occurrences of p in A with **1**.

The formulas $A[0/p]$ and $A[1/p]$ are called **restrictions** of A .

For example:

- If A is $\neg(p \rightarrow \neg q)$, then $A[0/p] \equiv \neg(0 \rightarrow \neg q) \equiv \neg 1 \equiv 0$
 $A[1/p] \equiv \neg(1 \rightarrow \neg q) \equiv \neg \neg q \equiv q$
 $A[0/q] \equiv \neg(p \rightarrow \neg 0) \equiv \neg 1 \equiv 0$
 $A[1/q] \equiv \neg(p \rightarrow \neg 1) \equiv \neg \neg p \equiv p$
- If A is $(p \vee r) \wedge (r \rightarrow (\neg p \wedge t))$, then $A[1/r] \equiv (p \vee 1) \wedge (1 \rightarrow (\neg p \wedge t))$
 $\equiv 1 \wedge (\neg p \wedge t) \equiv \neg p \wedge t$
- If p does not occur in A , then both $A[0/p]$ and $A[1/p]$ are just A .
- **Shannon expansion:** For all formulas A and variables p ,

$$A \equiv (p \wedge A[1/p]) \vee (\neg p \wedge A[0/p])$$

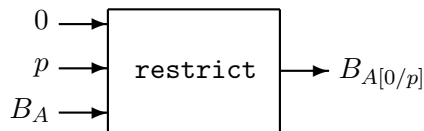
The algorithm `restrict`

Suppose we have a reduced OBDD B_A , representing a formula A , and a propositional variable p . Then:

- `restrict(0, p, BA)` **computes the reduced OBDD** $B_{A[0/p]}$ representing $A[0/p]$.
- `restrict(1, p, BA)` **computes the reduced OBDD** $B_{A[1/p]}$ representing $A[1/p]$.

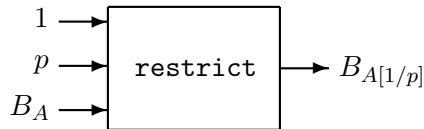
`restrict(0, p, BA)`: For each node n in B_A labelled by p ,

- redirect incoming edges to $lo(n)$ (special case: if n is the root, then $lo(n)$ becomes the new root),
- remove n , together with its outgoing edges, and
- remove any other nodes and edges that are no longer reachable from the root.
- Then apply reductions if necessary.



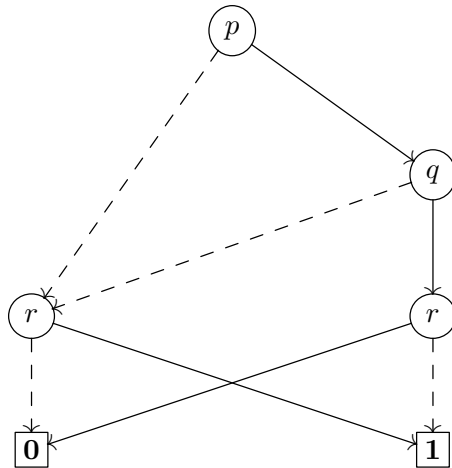
`restrict(1, p, BA)`: For each node n in B_A labelled by p ,

- redirect incoming edges to $hi(n)$ (special case: if n is the root, then $hi(n)$ becomes the new root),
- remove n , together with its outgoing edges, and
- remove any other node and edges that are no longer reachable from the root.
- Then apply reductions if necessary.



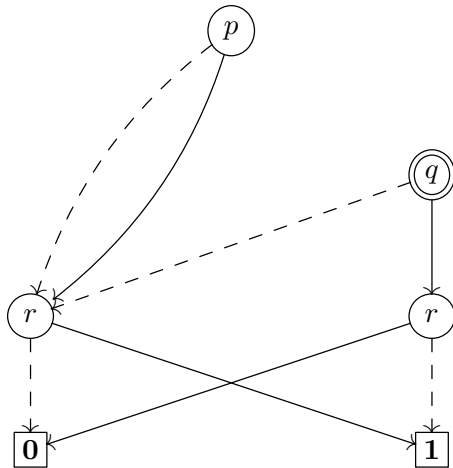
Example: $\text{restrict}(\mathbf{0}, q, B_A)$

Let B_A be the following OBDD. Compute $\text{restrict}(\mathbf{0}, q, B_A)$.



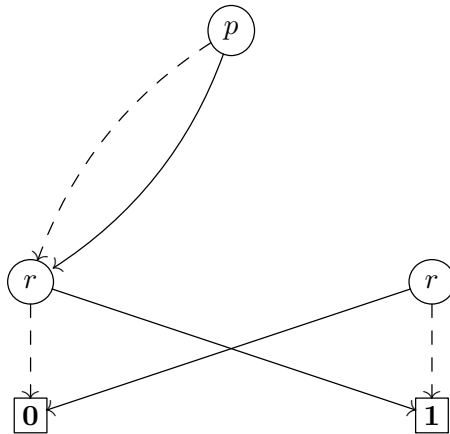
Example: $\text{restrict}(0, q, B_A)$ (cont.)

Step 1: For each node n labelled by q , redirect incoming edges to $lo(n)$.



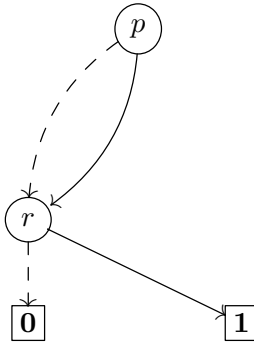
Example: $\text{restrict}(0, q, B_A)$ (cont.)

Step 2: For each node n labelled by q , remove n , together with its outgoing edges.

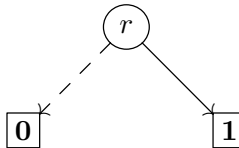


Example: $\text{restrict}(0, q, B_A)$ (cont.)

Step 3: Remove nodes and edges not reachable from the root.

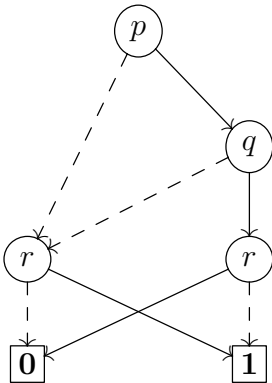


Step 4: Apply reductions.

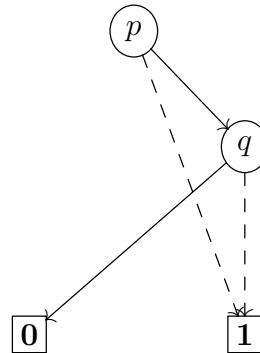


Example: $\text{restrict}(1, r, B_A)$

B_A



$\text{restrict}(1, r, B_A)$



The algorithm exists

Given a formula A and a propositional variable p , we define

$$\boxed{\exists p A} \text{ as a shorthand for } \boxed{A[0/p] \vee A[1/p]}.$$

For example:

- $\exists p (p \wedge q) \equiv (0 \wedge q) \vee (1 \wedge q) \equiv 0 \vee q \equiv q$
- $\exists q ((p \vee q) \rightarrow r) \equiv ((p \vee 0) \rightarrow r) \vee ((p \vee 1) \rightarrow r) \equiv (p \rightarrow r) \vee (1 \rightarrow r)$
 $\equiv (p \rightarrow r) \vee r$

Task: Given an OBDD B_A and p , compute the reduced OBDD $B_{\exists p A}$ for $\exists p A$.

$$\boxed{\text{exists}(p, B_A) : \text{apply}(\vee, \text{restrict}(0, p, B_A), \text{restrict}(1, p, B_A))}$$